Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского
Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИип2
Ковалев Б. В.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
  3.1. Описание структуры программы 6
  3.2. Описание структур данных 6
  3.3. Описание алгоритмов 6
4. Результаты экспериментов и тестирования 9
Заключение 11
Литература 12
Приложение 13

Введение

В рамках лабораторной работы №1 рассматривается реализация двух пользовательских классов: TBitField (битовое поле) и TSet (множество на основе битового поля).

Битовое поле — это компактная структура данных для хранения последовательности битов. В отличие от массива bool (по 1 байту на элемент), битовое поле упаковывает 32 бита в одно 32-битное число unsigned int, что дает экономию памяти в 8 раз.

Множество — математический объект, хранящий уникальные элементы и поддерживающий операции объединения, пересечения, разности и дополнения. Если элементами множества являются целые числа из диапазона 0..N-1, его удобно представить характеристическим вектором — битовым полем длины N, где бит i равен 1, если число i входит в множество.

Цель работы: освоить принципы ООП (инкапсуляцию, перегрузку операторов, правило трех) и научиться писать автотесты с использованием Google Test.

1. Постановка задачи

Требуется реализовать на языке C++ два класса:

1. Класс TBitField — битовое поле со следующими операциями:
  - конструктор по длине, конструктор копирования, деструктор;
  - методы GetLength, SetBit, ClrBit, GetBit;
  - перегруженные операторы ==, !=, =, |, &, ~;
  - операторы ввода/вывода в поток.

2. Класс TSet — множество на основе TBitField:
  - конструктор по мощности, конструктор копирования, конструктор преобразования из TBitField;
  - оператор преобразования к TBitField;
  - методы GetMaxPower, InsElem, DelElem, IsMember;
  - теоретико-множественные операции: + (с элементом и с множеством), - (с элементом), * (пересечение), ~ (дополнение);
  - операторы сравнения, присваивания и ввода/вывода.

Объявления классов должны точно соответствовать официальным заголовкам из папки includes/Lab1 репозитория. Реализация выносится в отдельные .cpp-файлы.

Дополнительно требуется написать набор тестов Google Test, покрывающих основной функционал и краевые случаи обоих классов.

2. Руководство пользователя

Данная работа не является консольным приложением с интерактивным вводом. Всё взаимодействие происходит через автотесты Google Test.

Запуск тестов:

1. Собрать проект:
   $ cd KovalevB/Lab1
   $ make
2. Запустить исполняемый файл тестов:
   $ ./run_tests
3. На экране отобразится список выполненных тестов и итог:
   [==========] 39 tests from 2 test suites ran. (0 ms total)
   [PASSED] 39 tests.

Если какой-то тест не прошел — будет выведено сообщение об ошибке с указанием файла, номера строки, ожидаемого и фактического значений.

Также тесты можно запускать через панель Testing в VS Code (расширение C++ TestMate) или через обозреватель тестов в Visual Studio.

3. Руководство программиста

3.1. Описание структуры программы

Проект состоит из следующих модулей:
- tbitfield.h — объявление класса TBitField (официальный заголовок из includes/Lab1, не дублируется в проекте).
- tbitfield.cpp — реализация методов класса TBitField.
- tset.h — объявление класса TSet (официальный заголовок из includes/Lab1).
- tset.cpp — реализация методов класса TSet.
- test_tbitfield.cpp — набор тестов Google Test для класса TBitField.
- test_tset.cpp — набор тестов Google Test для класса TSet.

3.2. Описание структур данных

Класс TBitField:
Поля:
- int BitLen — логическая длина (число доступных битов);
- int MemLen — размер массива pMem (сколько элементов TELEM нужно);
- TELEM* pMem — динамический массив элементов типа unsigned int.
Приватные методы:
- GetMemIndex(n) — индекс элемента для бита n: n / 32;
- GetMemMask(n) — маска бита: 1u << (n % 32).
Бит с номером n хранится в элементе pMem[n / 32] на позиции n % 32. Это дает плотную упаковку: 32 бита в одном unsigned int.

Класс TSet:
Поля:
- int MaxPower — мощность универсума (числа 0..MaxPower-1);
- TBitField BitField — характеристический вектор множества.
Бит i вектора равен 1, если число i входит в множество, и 0 иначе. Все теоретико-множественные операции сводятся к побитовым операциям над характеристическими векторами.

3.3. Описание алгоритмов

Установка бита (SetBit):

1. Проверить, что 0 <= n < BitLen (иначе выбросить std::out_of_range).
2. Вычислить индекс элемента: idx = n / 32.

3. Вычислить маску: mask = 1u << (n % 32).
4. Выполнить побитовое ИЛИ: pMem[idx] |= mask.

Очистка бита (ClrBit):

1. Проверить границы индекса n.
2. Вычислить idx = n / 32, mask = 1u << (n % 32).
3. Выполнить pMem[idx] &= ~mask (обнуляем нужный бит, остальные не трогаем).

Чтение бита (GetBit):

1. Проверить границы индекса n.
2. Вернуть (pMem[idx] & mask) != 0.

Операция ИЛИ (operator|):

1. Взять максимум длин двух полей.
2. Создать новое поле этой длины.
3. Скопировать элементы первого поля и применить побитовое ИЛИ с элементами второго поля.

Операция И (operator&):

1. Взять максимум длин двух полей и создать новое поле.
2. На минимальной длине массивов minMemLen вычислить temp.pMem[i] = pMem[i] & bf.pMem[i].

Инверсия (operator~):

1. Создать результат той же длины BitLen.
2. Для каждого элемента: temp.pMem[i] = ~pMem[i].
3. В последнем элементе затереть «лишние» старшие биты, выходящие за пределы BitLen, через маску (1u << lastBits) - 1.

Конструктор копирования и operator=:

1. В конструкторе копирования выделить массив MemLen и скопировать элементы через std::copy.
2. В operator= проверить самоприсваивание (this == &bf), при неравенстве MemLen перевыделить массив pMem, обновить BitLen и скопировать данные.

Объединение и пересечение множеств (operator+, operator*):

1. Проверить равенство мощностей MaxPower == s.MaxPower (иначе выбросить std::invalid_argument).

2. Применить побитовое ИЛИ (|) или И (&) к характеристическим векторам и вернуть новое множество.

Дополнение, добавление, удаление и проверка элемента:

1. Дополнение (operator~) возвращает TSet(~BitField).
2. InsElem, DelElem и IsMember вызывают соответственно SetBit, ClrBit и GetBit для номера элемента в BitField.

Операторы ввода и вывода (operator>>, operator<<):

1. Ввод TBitField и TSet предварительно очищает состояние объекта (перезаписывает его), пропускает пробелы и считывает биты '0'/'1' или список элементов в фигурных скобках { ... }.
2. Вывод TBitField печатает битовую строку, вывод TSet печатает элементы множества через пробел в формате { i1 i2 ... }.

4. Результаты экспериментов и тестирования

Написано 39 тестов Google Test, разбитых на два набора.

Набор TBitFieldTest (22 теста):

- can_create_bitfield_with_positive_length — создание поля положительной длины;
- can_get_length — проверка корректности длины;
- throws_when_create_bitfield_with_negative_or_zero_length — исключение при длине <= 0;
- new_bitfield_is_initialized_to_zero — обнуление при создании;
- can_set_and_get_bit_across_telem_boundaries — работа на границах TELEM;
- setting_bit_twice_keeps_it_set — повторная установка бита;
- can_clear_bit — сброс бита;
- clearing_zero_bit_keeps_it_zero — сброс нулевого бита;
- throws_when_set_bit_out_of_bounds — исключение при SetBit вне границ;
- throws_when_clear_bit_out_of_bounds — исключение при ClrBit вне границ;
- throws_when_get_bit_out_of_bounds — исключение при GetBit вне границ;
- copy_constructor_creates_independent_deep_copy — глубокое копирование;
- can_assign_bitfields_of_equal_and_different_sizes — проверка operator=;
- self_assignment_is_safe — безопасность самоприсваивания;
- compare_equal_and_non_equal_bitfields — операторы == и !=;
- bitwise_OR_on_equal_and_different_sizes — операция | (объединение битов);
- bitwise_AND_on_equal_and_different_sizes — операция & (пересечение битов);
- bitwise_NOT_on_exact_telem_multiple — операция ~ на длине, кратной 32;
- bitwise_NOT_clears_tail_on_non_multiple_size — очистка «хвоста» при ~;
- stream_output_and_input_roundtrip — ввод и вывод битовой строки;
- stream_input_overwrites_object_on_short_input — перезапись при коротком вводе;
- stream_input_stops_on_invalid_character — остановка ввода на некорректном символе.

Набор TSetTest (17 тестов):

- can_create_set_with_positive_max_power — проверка мощности;
- throws_when_create_set_with_negative_or_zero_max_power — исключение при мощности <= 0;
- can_copy_set — копирование множества;

- can_convert_between_tset_and_tbitfield — преобразование TSet <-> TBitField;
- can_insert_check_and_delete_elements_at_boundaries — вставка, проверка и удаление на границах;
- throws_when_insert_delete_or_check_out_of_bounds — исключения при выходе за границы;
- can_assign_sets_of_different_sizes — присваивание множеств разной мощности;
- compare_sets_of_equal_and_different_universes — сравнение множеств;
- operator_plus_and_minus_element_do_not_modify_original_set — операторы + и - с элементом;
- operator_plus_and_minus_element_throw_out_of_bounds — исключения в + и - с элементом;
- can_union_two_sets_from_same_universe — объединение двух множеств;
- throws_when_union_sets_from_different_universes — исключение при объединении разных универсумов;
- can_intersect_two_sets_from_same_universe — пересечение двух множеств;
- throws_when_intersect_sets_from_different_universes — исключение при пересечении разных универсумов;
- can_complement_set_with_large_power — дополнение множества;
- stream_output_and_input_roundtrip_with_multidigit_elements — потоковый ввод и вывод;
- stream_input_handles_empty_set_and_overwrites_previous_state — чтение пустого множества и перезапись.

Результат: все 39 тестов PASSED за ~0 ms. Это подтверждает корректность реализации как основного функционала, так и краевых случаев.

Заключение

В ходе лабораторной работы были реализованы два класса:
1. TBitField — битовое поле с полным набором битовых операций, корректной работой с памятью (конструктор копирования, деструктор, оператор присваивания — правило трех) и обработкой краевых случаев (выход за границы, «лишние» биты, перезапись при вводе).
2. TSet — множество на основе характеристического вектора. Все теоретико-множественные операции (объединение, пересечение, дополнение, разность) сведены к побитовым операциям с проверкой совпадения универсумов, что дает высокую производительность и компактность.

Освоены:
- принципы инкапсуляции и перегрузки операторов в C++;
- правило трех при управлении динамической памятью;
- дружественные функции для перегрузки операторов ввода-вывода;
- фреймворк Google Test для написания автотестов;
- базовые приемы работы с Git и GitHub.

Литература

Список источников, используемых при выполнении работы:
1. Страуструп Бьерн. Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Документация Google Test: https://google.github.io/googletest/

Приложение

```
[==========] Running 39 tests from 2 test suites.
[----------] 22 tests from TBitFieldTest (0 ms total)
[----------] 17 tests from TSetTest (0 ms total)
[==========] 39 tests from 2 test suites ran. (0 ms total)
[  PASSED  ] 39 tests.
```